

第 11 讲 实验设计

Forward Propagation 与 Computational Graph

实验总目标

把 “Forward” 从一个术语变成可观察、可验证的计算过程：显式保存中间值，记录 Shape，改变输入，观察影响怎样沿依赖路径传播。

一、研究问题

- RQ1: 一个 Forward Pass 是否可以完全拆成一系列确定的中间变量？
- RQ2: 改变上游输入后，变化是否沿其依赖路径传到下游？
- RQ3: ReLU 是否可能让上游数值变化在某个区间内不再影响下游？
- RQ4: Batch Forward 是否只是 “同一套参数和计算规则同时处理多条样本” ？
- RQ5: NumPy 手工 Forward 与 PyTorch 层级 Forward 是否能得到一致结果？

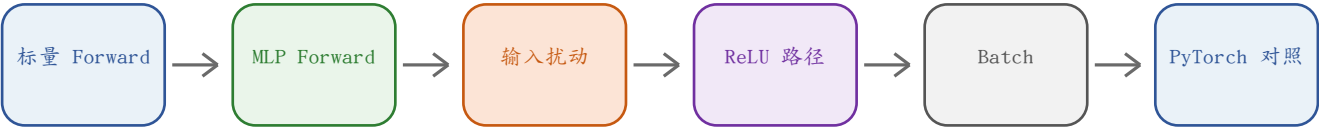
二、实验假设

编号	假设
H1	显式逐节点计算得到的最终输出与封装后的模型输出一致。
H2	改变 <code>x[0]</code> 后，只会通过依赖 <code>x[0]</code> 的路径改变后续中间量。
H3	当 ReLU 输入仍为负数时，即使输入数值变化，ReLU 输出仍保持 0。
H4	Batch 只增加批维，Linear 的特征变换规则保持不变。
H5	参数完全对齐后，NumPy 和 PyTorch Forward 只存在浮点误差。

三、变量与观测指标

类型	内容
自变量	输入 x ; 实验 3 中对 $x[0]$ 加 0.1; 实验 4 中改变 ReLU 的输入 z
固定量	$W1$ 、 $b1$ 、 $W2$ 、 $b2$; 网络结构 $3 \rightarrow 4 \rightarrow 2$
中间变量	$z1$ 、 $a1$
输出变量	y
观测指标	每个节点实际数值、Shape、Delta $z1$ / Delta $a1$ / Delta y 、NumPy-PyTorch 最大误差

四、实验流程

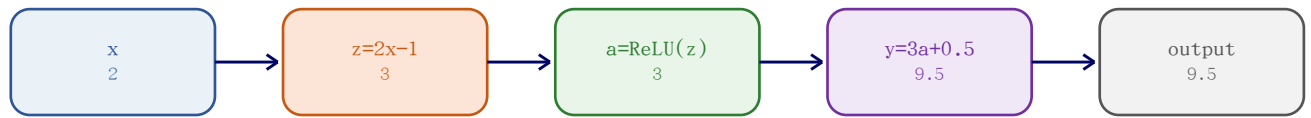


控制变量思想
实验 3 只修改一个输入分量，其余参数全部不变；这样才能把输出变化归因于这一处输入扰动。

五、实验 1：最小标量计算图

输入 $x=2$ ，依次计算 $z=2x-1$ 、 $a=\text{ReLU}(z)$ 、 $y=3a+0.5$ 。

一个最小 Forward Computational Graph



预期： $x=2 \rightarrow z=3 \rightarrow a=3 \rightarrow y=9.5$ 。这个实验用于确认“Forward 就是按依赖顺序求值”。

六、实验 2：两层 MLP 的中间量

沿用第 10 讲的固定参数，显式保存 `z1`、`a1`、`y`。

```
z1 = x @ W1 + b1
a1 = np.maximum(0.0, z1)
y = a1 @ W2 + b2
```

预期： $z1=[1,3,-1,-3]$ ， $a1=[1,3,0,0]$ ， $y=[2.7,0.2]$ 。

七、实验 3：输入扰动如何向下游传播

把 $x[0]$ 从 1.0 改成 1.1，然后重新 Forward。记录：

观测量	含义
Delta z_1	第一层哪些节点首先变化
Delta a_1	经过 ReLU 后哪些变化被保留 / 截断
Delta y	最终输出变化

重要限制

Delta y / 0.1 目前也不要直接称为“梯度”。这是有限差分的直观观察；真正的导数与 Gradient 会在后面按极限 / 局部变化率来定义。

八、实验 4：ReLU 路径截断

依次测试 $z=-1.0$ 、 -0.5 、 0.5 、 2.0 ，并计算 $y=2*\text{ReLU}(z)$ 。重点观察：前两个 z 不同，但 ReLU 后都变成 0，因此 y 相同。

九、实验 5：Batch Forward

构造 3 条样本 X .shape=(3,3)，同时通过同一套 $W1$ 、 $b1$ 、 $W2$ 、 $b2$ 。

$$(3,3) @ (3,4) \rightarrow (3,4) \rightarrow \text{ReLU} \rightarrow (3,4) @ (4,2) \rightarrow (3,2)$$

目的：验证 Batch 不是创建三套模型，而是让同一套参数同时处理三条样本。

十、实验 6：PyTorch 对照

创建 `nn.Sequential(Linear(3,4), ReLU(), Linear(4,2))`，然后把参数手动复制成与 NumPy 一致。不要只运行 `model(x)`，而是逐层调用并打印 `z1_t`、`a1_t`、`y_t`。

验收

如果 NumPy 与 PyTorch 的最终输出最大误差接近 0，说明你对 Forward 计算规则和 PyTorch Linear 权重存储方向的理解是一致的。

十一、实际运行结果

```
=====
1 Forward
=====
x = 2.0
z = 2*x - 1 = 3.0
a = ReLU(z) = 3.0
y = 3*a + 0.5 = 9.5
x -> z -> a -> y

=====
2 MLP Forward 10
=====
x = [ 1.  2. -1.] shape = (3,)
z1 = [ 1.  3. -1. -3.] shape = (4,)
a1 = [1.  3.  0.  0.] shape = (4,)
y = [2.7 0.2] shape = (2,)
Forward x -> z1 -> a1 -> y

=====
3 Forward Gradient
=====
x = [ 1.  2. -1.]
x = [ 1.1  2. -1. ]
Delta z1 = [ 0.1  0. -0.1  0.05]
Delta a1 = [0.1 0.  0.  0. ]
Delta y = [ 0.1 -0.1]
Forward Gradient

=====
4 ReLU Forward
=====
z=-1.0 -> ReLU(z)=0.0 -> y=0.0
z=-0.5 -> ReLU(z)=0.0 -> y=0.0
z= 0.5 -> ReLU(z)=0.5 -> y=1.0
z= 2.0 -> ReLU(z)=2.0 -> y=4.0

=====
5 Batch Forward -
=====
X.shape = (3, 3)
Z1.shape = (3, 4)
A1.shape = (3, 4)
Y.shape = (3, 2)
Y =
[[ 2.7  0.2 ]
 [ 0.95 0.45]
 [ 6.2 -2.3 ]]

=====
6 PyTorch nn.Sequential Forward
=====
PyTorch z1 = [ 1.  3. -1. -3.]
PyTorch a1 = [1.  3.  0.  0.]
PyTorch y = [2.7 0.2]
NumPy y = [2.7 0.2]
max|PyTorch-NumPy| = 4.7683715642676816e-08

=====
```

十二、结果解释

标量链严格得到 $2 \rightarrow 3 \rightarrow 3 \rightarrow 9.5$ ，说明 Forward 就是按计算依赖逐节点求值。

固定 MLP 得到与第 10 讲相同的 $z1$ 、 $a1$ 、 y ，说明“结构不变 + 参数不变 + 输入不变”时 Forward 是确定的。

输入 $x[0]$ 增加 0.1 后， $z1$ 的多个分量变化，因为同一个输入特征连接到多个隐藏神经元。

某些 z 的变化经过 ReLU 后可能消失，这体现非线性对 Forward 信息流的筛选。

Batch 输出 Shape 与矩阵规则一致，说明批处理只是把相同计算向量化。

NumPy 与 PyTorch 输出只存在浮点误差，说明框架封装并没有改变底层数学。

实验结论

H1-H5 均得到支持。下一阶段学习 Loss 和 Gradient

后，我们会把同一张图从“只看数值流”升级为“同时看梯度流”。